

第6章 分支限界法

(Branch and Bound)

学习要点

- 理解分支限界法的剪枝搜索策略。
- 掌握分支限界法的算法框架
 - (1) 队列式(FIFO)分支限界法
 - (2) 优先队列式分支限界法
- 通过应用范例学习分支限界法的设计策略。
 - (1)单源最短路径问题；(2)装载问题；(3)0-1背包问题；
 - (4)最大团问题；(5)旅行售货员问题；(6)批处理作业调度问题

6.1 分支限界法的基本思想

■分支限界法与回溯法

■求解目标：**回溯法**的求解可以找出解空间树中满足约束条件的所有解。而**分支限界法**的求解目标则是找出满足约束条件的一个解，或是在满足约束条件的解中找出在某种意义下的最优解。

■搜索方式的不同：**回溯法**以**深度优先的方式**搜索解空间树，而**分支限界法**则以**宽度优先或以最小耗费优先的方式**搜索解空间树。

6.1 分支限界法的基本思想

■常见的两种分支限界法

(1) 队列式(FIFO)分支限界法

按照队列先进先出 (FIFO) 原则选取下一个节点为扩展节点。

数据结构：队列

(2) 优先队列式分支限界法

按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。

数据结构：堆

最大优先队列：最大堆，体现最大效益优先

最小优先队列：最小堆，体现最小费用优先

队列式(FIFO)分支限界法

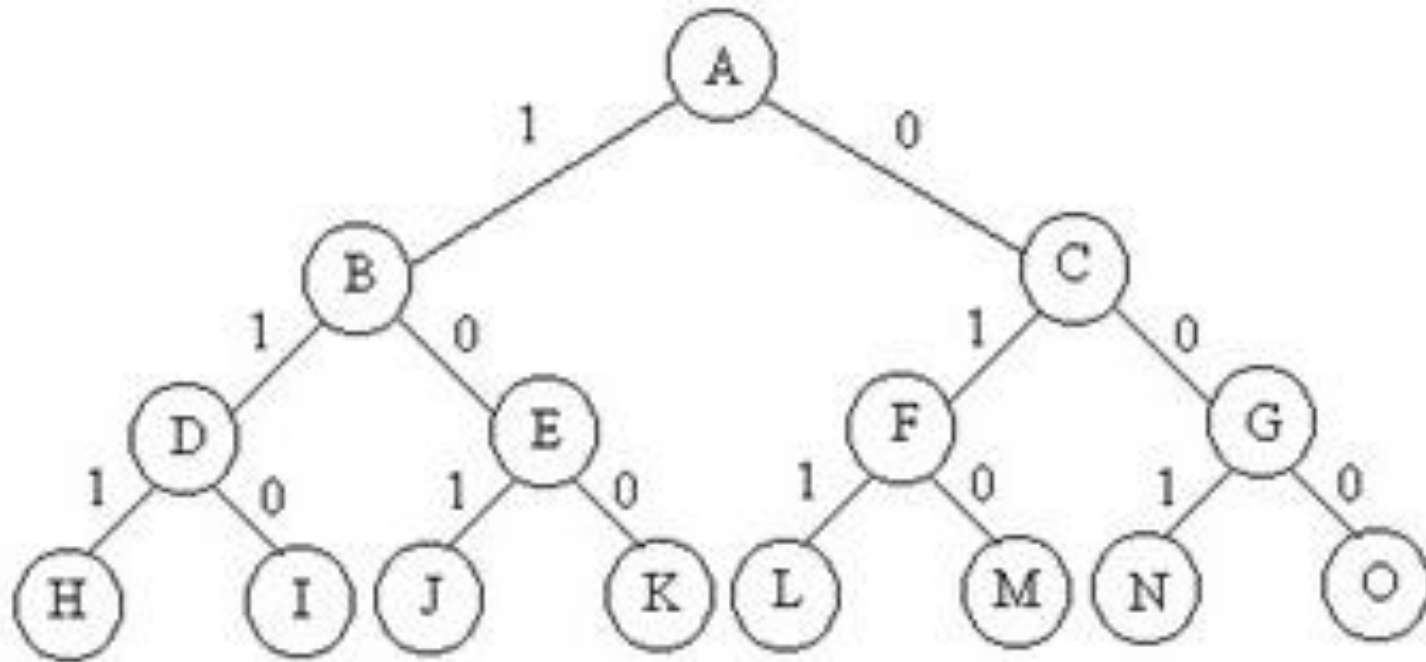
- 搜索策略：一开始，根节点是唯一的活节点，根节点入队。从活节点队列中取出根节点后，作为当前扩展节点。对当前扩展节点，**先从左到右地产生它的所有儿子，用约束条件检查**，把所有满足约束函数的儿子加入活节点队列中。再从活节点表中取出队首节点为当前扩展节点，……，直到**找到一个解或活节点队列为空**为止。

优先队列式分支限界法

- 搜索策略：对每一活结点计算一个**优先级**（某些信息的函数值）；并根据这些优先级，从当前活结点表中优先选择一个**优先级最高**（最有利）的结点作为扩展结点，使搜索朝着解空间树上有最优解的分支推进，以便尽快地找出一个最优解。再从活结点表中下一个优先级别最高的结点为当前扩展结点，……，直到找到一个解或活结点队列为空为止。

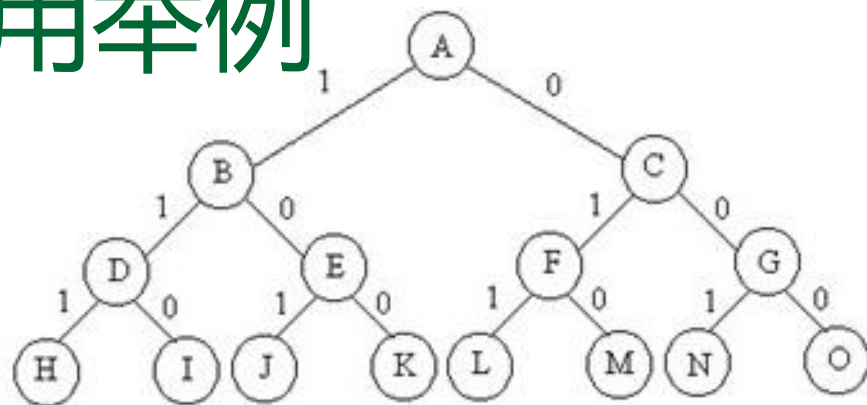
分支限界法搜索应用举例

- 0-1背包问题，当 $n=3$ 时， $w=\{16,15,15\}$,
 $p=\{45,25,25\}$, $c=30$



分支限界法搜索应用举例

0-1背包问题, 当 $n=3$ 时, $c=30$
 $w=\{16,15,15\}$, $p=\{45,25,25\}$



- **队列式分支限界法**(处理法则:先进先出):
 $\{\} \rightarrow \{A\} \rightarrow \{B, C\} \rightarrow \{C, \textcolor{red}{D}, E\} \rightarrow \{E, F, G\} \rightarrow$
 $\{F, G, \textcolor{red}{J}, K\} \rightarrow \{G, K, L, M\} \rightarrow \{\textcolor{yellow}{K}, \textcolor{red}{L}, M, N, O\}$
- **优先队列式分支限界法**(处理法则:价值大者优先)
 $\{\} \rightarrow \{A\} \rightarrow \{B, C\} \rightarrow \{C, E, \textcolor{red}{D}\} \rightarrow \{F, K, G\} \rightarrow$
 $\{\textcolor{red}{L}, K, M, G\}$

- 通过应用范例学习分支限界法的设计策略。

- (1) **装载问题**;

- (2) 0-1背包问题;

子集树

- (3) 最大团问题;

- (4) 旅行售货员问题;

- (5) 单源最短路问题;

排列树

- (6) 批处理作业调度问题

6.3 装载问题

1. 问题描述

有一批共个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且
$$\sum_{i=1}^n w_i \leq c_1 + c_2$$

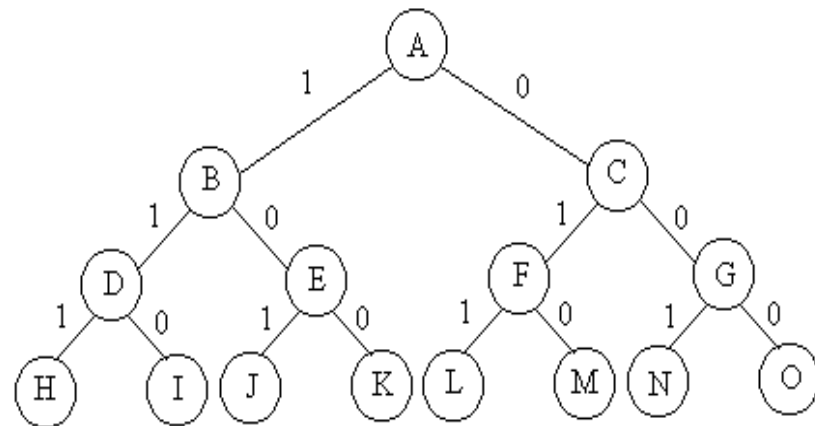
装载问题要求确定是否有一个合理的装载方案可将这个集装箱装上这2艘轮船。如果有，找出一种装载方案。

容易证明：如果一个给定装载问题有解，则采用下面的策略可得到最优装载方案。

- (1)首先**将第一艘轮船尽可能装满**；
- (2)将剩余的集装箱装上第二艘轮船。

6.3 装载问题

解空间：子集树



可行性约束函数(选择当前元素): $\sum_{i=1}^n w_i x_i \leq c_1$

加入队列的算法EnQueue

EnQueue(Q, wt, bestw, i, n) //将活结点加入到活结点队列Q中

{ if (i == n) { // 可行叶子结点

if (wt > bestw) bestw = wt; }

else

Q.Add(wt); // 非叶子节点

}

6.3 装载问题

2. 队列式分支限界法

```
while (true) {  
    // 检查左儿子结点  
    if ( $Ew + w[i] \leq c$ ) //  $x[i] = 1$   
        EnQueue(Q,  $Ew + w[i]$ , bestw, i, n);  
    // 右儿子结点总是可行的  
    EnQueue(Q,  $Ew$ , bestw, i, n); //  $x[i] = 0$   
    Q.Delete(Ew); // 取下一扩展结点  
    if ( $Ew == -1$ ) { // 同层结点尾部  
        if (Q.IsEmpty()) return bestw;  
        Q.Add(-1); // 同层结点尾部标志  
        Q.Delete(Ew); // 取下一扩展结点  
        i++; // 进入下一层    } }
```

■在算法的while循环中，首先检测当前扩展结点的左儿子结点是否为可行结点。如果是则将其加入到活结点队列中。然后将其右儿子结点加入到活结点队列中(右儿子结点一定是可行结点)。2个儿子结点都产生后，当前扩展结点被舍弃。

■活结点队列中的队首元素被取出作为当前扩展结点，由于队列中每一层结点之后都有一个尾部标记-1，故在取队首元素时，活结点队列一定不空。当取出的元素是-1时，再判断当前队列是否为空。如果队列非空，则将尾部标记-1加入活结点队列，算法开始处理下一层的活结点。

6.3 装载问题

2. 队列式分支限界法

```
while (true) {
```

```
    // 检查左儿子结点
```

```
    if (Ew + w[i] <= c) // x[i] = 1
```

```
        EnQueue(Q, Ew + w[i], bestw, i, n);
```

```
    // 右儿子结点总是可行的
```

```
    EnQueue(Q, Ew, bestw, i, n);
```

```
    Q.Delete(Ew);
```

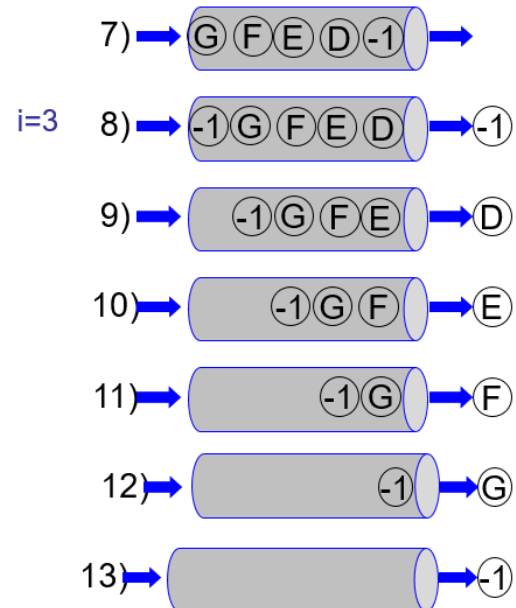
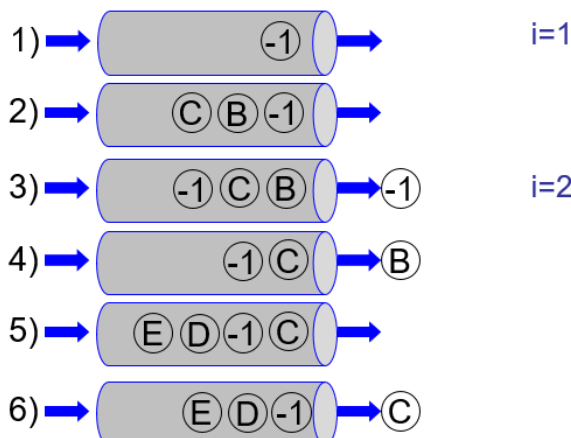
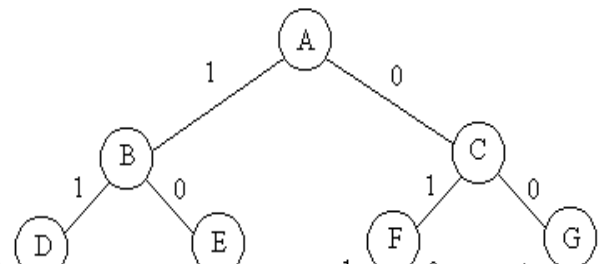
```
    if (Ew == -1) {
```

```
        if (Q.IsEmpty()) return bestw;
```

```
        Q.Add(-1);
```

```
        Q.Delete(Ew); // 取下一扩展结点
```

```
        i++;}           // 进入下一层 }
```



算法的改进?

6.3 装载问题

3. 算法的改进

// 检查左儿子结点

Type wt = Ew + w[i]; // 左儿子结点的重量

if (wt <= c) { // 可行结点

if (wt > bestw) bestw = wt;

// 加入活结点队列

if (i < n) Q.Add(wt);

}

// 检查右儿子结点

if (Ew + r > bestw && i < n)

Q.Add(Ew); // 可能含最优解

Q.Delete(Ew); // 取下一扩展结点

右儿子剪枝

提前更新bestw

6.3 装载问题

为什么回溯不需要？

4. 构造最优解

为了在算法结束后能方便地构造出与最优值相应的最优解，算法必须存储相应子集树中从活结点到根结点的路径。为此目的，可在每个结点处设置指向其父结点的指针，并设置左、右儿子标志。

```
class QNode
```

```
{QNode *parent; // 指向父结点的指针
```

```
    bool LChild;    // 左儿子标志
```

```
    Type weight;    // 结点所相应的载重量
```

```
}
```

```
QNode<Type> *b
```

```
b= new QNode<Type>
```

```
b->weight = wt;
```

```
b->parent = E;
```

```
b->Lchild =ch;
```

6.3 装载问题

4. 构造最优解

找到最优值后，可以根据parent回溯到根节点，找到最优解。

// 构造当前最优解

```
for (int j = n - 1; j > 0; j--) {  
    bestx[j] = bestE->LChild;  
    bestE = bestE->parent;  
}
```

6.3 装载问题

5. 优先队列式分支限界法

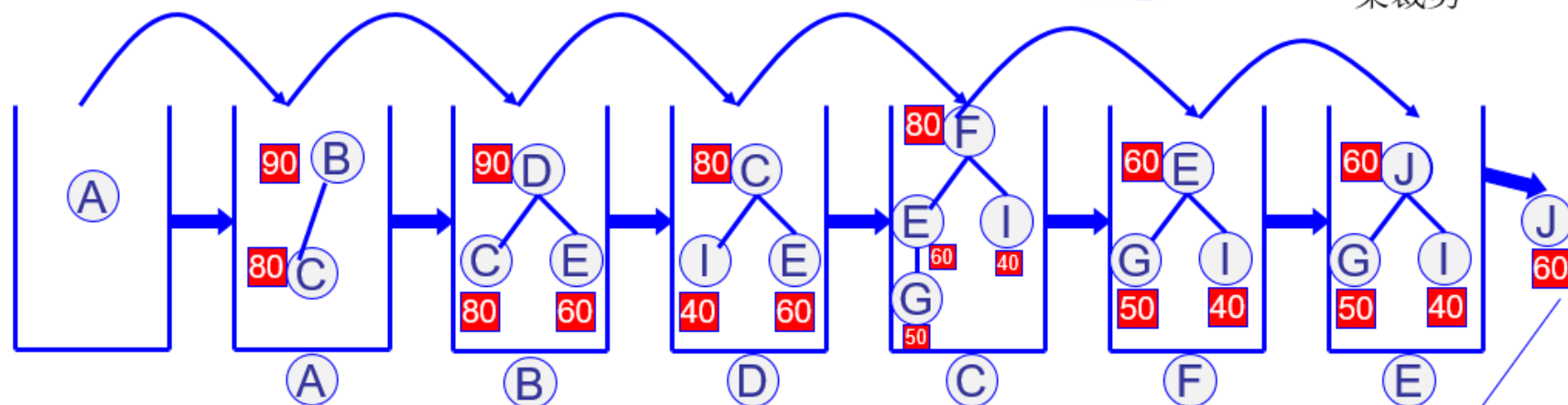
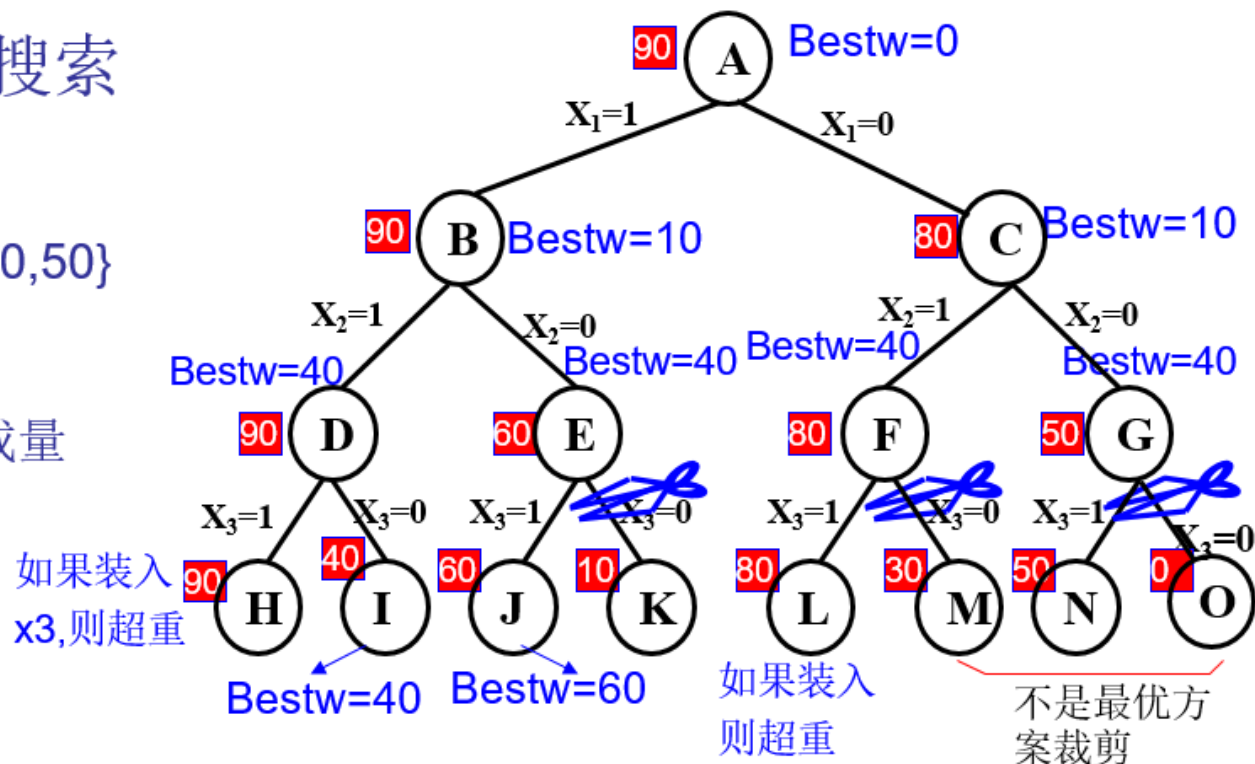
- 解装载问题的优先队列式分支限界法用最大优先队列存储活结点表。活结点 x 在优先队列中的优先级定义为**载重上界**，即从根结点到结点 x 的路径所相应的载重量再加上剩余集装箱的重量之和。
- 优先队列中优先级最大的活结点成为下一个扩展结点。以结点 x 为根的子树中所有结点相应的路径的载重量不超过它的优先级。子集树中叶结点所相应的载重量与其优先级相同。

优先队列+限界搜索

物品的重量 $W=\{10,30,50\}$

$c_1=60$

方框数字为装载上界,
 $Bestw$ 为当前最优装载量



扩展层数4(叶子), 算法结束

- 通过应用范例学习分支限界法的设计策略。

- (1) 装载问题;

- (2) 0-1背包问题;

- (3) 最大团问题;

- (4) 旅行售货员问题;

- (5) 单源最短路问题;

- (6) 批处理作业调度问题

6.5 0-1背包问题

■ 问题描述

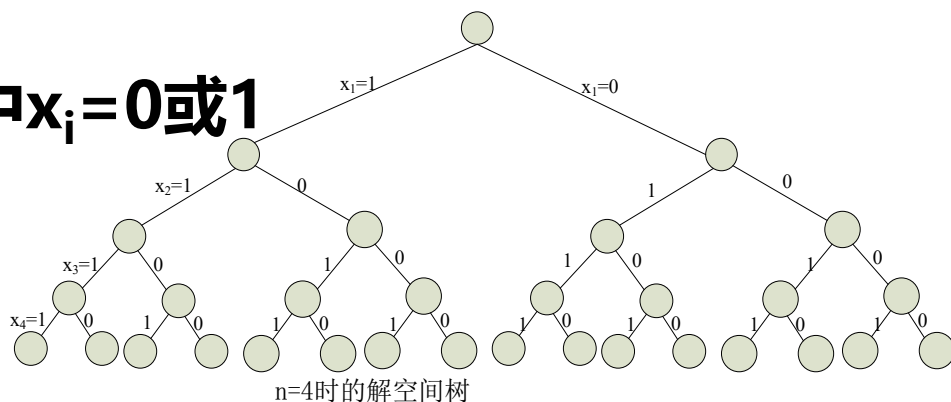
给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 W 。一个物品要么全部装入背包，要么全部不装入背包，不允许部分装入。装入背包的物品的总重量不超过背包的容量。问应如何选择装入背包的物品，使得装入背包中的物品总价值最大？

■ 定义问题的解空间

解的形式 $(x_1, x_2, \dots, x_n)$ ，其中 $x_i = 0$ 或 1

$x_i = 0$: 第 i 个物品不装入背包

$x_i = 1$: 第 i 个物品装入背包



6.5 0-1背包问题

优先队列分支限界法算法的思想

- 首先，要对输入数据进行预处理，将各物品依其单位重量价值从大到小进行排列。
- **节点的优先级 $\text{bound}(X)$** ：已装袋的物品价值加上剩下的最大单位重量价值的物品装满剩余容量的价值和。
- **搜索过程**：算法首先检查当前扩展结点的左儿子结点的可行性。如果该左儿子结点是可行结点，则将它加入到子集树和活结点优先队列中。当前扩展结点的右儿子结点一定是可行结点，仅当右儿子结点满足上界约束时才将它加入子集树和活结点优先队列。当扩展到叶节点时为问题的最优值。

0-1背包问题

算法描述:

初始化, 将物品按照单位价值从大到小的顺序排列
从第一个物品开始

左儿子加入后是否满足约束

将顶点*i*加入到最大优先队列中

修改最大价值, 修改上界条件

检查右儿子是否满足限界条件

将右儿子加入优先队列

修改上界条件

取下一扩展结点

当队列为空时, 输出当前记录的路径

6.5 0-1背包问题

■ 数据结构

准备工作：创建一个最大堆，来实现活结点优先队列

```
MaxHeap<HeapNode> H(1000); //定义最大堆容量为1000
```

堆中元素类型为HeapNode。子集树中节点类型为bbnode

```
class HeapNode
{
    private:
        Typep upprofit,           //节点的价值上界
            profit;               //节点所相应的价值
        Typew weight;             //节点所相应的重量
        int level;                //活节点在子集树中所处的层序号

        bbnode *ptr;              //指向活节点在子集中相应节点的指针
};
```

6.5 0-1背包问题

■将活节点插入到子集树和优先队列

```
AddLiveNode(Typep up,Typep cp,Typew cw,bool ch,int lev)
{
    bbnode *b = new bbnode; // 创建子集树中的新结点
    b->parent = E;    //E是当前扩展节点
    b->LChild = ch;

    HeapNode<Typep,Typew> N; // 创建堆中的新结点
    N.uprofit = up;        //节点的价值上界
    N.profit = cp;         //节点所相应的价值
    N.weight = cw;        //节点所相应的重量
    N.level = lev;        //活节点在子集树中所处的层序号
    N.ptr = b;            //指向活节点在子集中相应节点的指针
    H->Insert(N);
}
```

6.5 0-1背包问题

■ 上界函数Bound

```
while (i <= n && w[i] <= cleft) {    // n表示物品总数
    cleft -= w[i];                    // cleft为剩余空间
    b += p[i];                        // w[i]表示i所占空间
    i++;                             // p[i]表示i的价值
}
if (i <= n) b += p[i] / w[i] * cleft; // 装填剩余容量装满
背包
return b;                             // b为上界函数
```

6.5 0-1背包问题

分支限界搜索过程

```
while (i != n+1) { // 非叶结点
```

```
    // 检查当前扩展结点的左儿子结点
```

```
    if (cw + w[i] <= c) { // 左儿子结点为可行结点
```

```
        if (cp+p[i] > bestp) bestp = cp+p[i];
```

```
        AddLiveNode(up, cp+p[i], cw+w[i], true, i+1);
```

```
        //AddLiveNode将活结点插入到子集树和优先队列
```

```
    // 检查当前扩展结点的右儿子结点
```

```
    if (Bound(i+1) >= bestp) // 右子树可能含最优解
```

```
        AddLiveNode(Bound(i+1), cp, cw, false, i+1);
```

```
    // 取下一个扩展节点 (略) }
```

```
//构造当前最优解 (略)
```

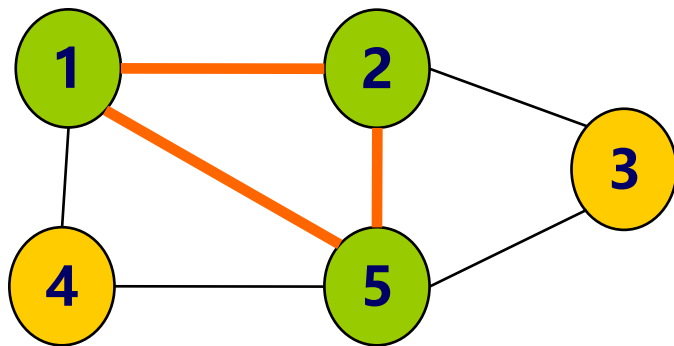
■ 通过应用范例学习分支限界法的设计策略。

- (1) 装载问题;
- (2) 0-1 背包问题;
- (3) 最大团问题;
- (4) 旅行售货员问题;
- (5) 单源最短路问题;
- (6) 批处理作业调度问题

6.6 最大团问题

1. 问题描述

- 最大团问题: 要求找出无向图 $G=(V, E)$ 的 n 个顶点集合 $\{1,2,3,...,n\}$ 的一个子集, 这个子集中的任意两个顶点在无向图 G 中都有边相连, 且包含顶点个数是最多的。



$\{1, 2, 5\}$ 、 $\{1, 4, 5\}$ 和
 $\{2, 3, 5\}$ 都是 G 的最大团

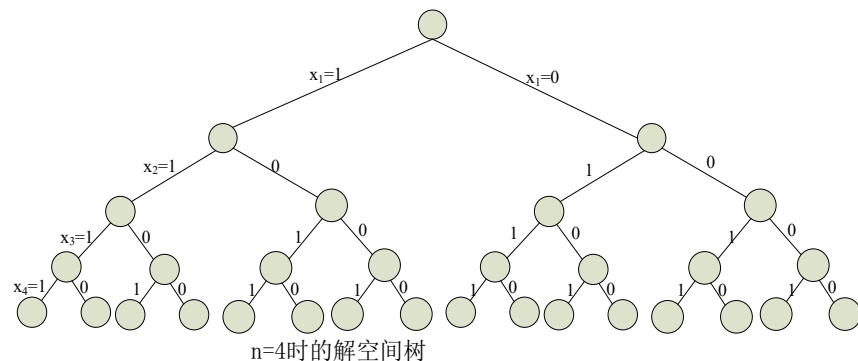
5.7 最大团问题

■ 定义问题的解空间

■ 子集树

■ 解的形式($x_1, x_2, \dots, x_n$)

■ 其中 $x_i=0$ 或 1 表示顶点 i 是否在当前团内



■ 约束条件：该顶点与当前团内每个顶点都有边相连。

■ 限界条件

■ $cn + rn > bestn$

■ cn : 当前已包含在顶点集中的顶点个数

■ rn : 剩余顶点个数($n-i$);

■ $bestn$: 当前已包含的最多的顶点个数

6.6 最大团问题

2. 优先级函数

- 优先级：当前已在团的顶点数加上还没有考察的顶点数，即 $cn+n-i$ 。
 - 每个活结点的优先顺序由 $cn+n-i$ 来确定。
 - 优先级计算简单，但是这个裁剪效果不是最好

6.6 最大团问题

3. 算法思想

- 以根结点作为初始扩展结点，其 cn 的值为0。
- 算法在扩展内部结点时，首先考察其左儿子结点。如果满足约束条件(顶点 i 与当前团中其它顶点之间都有边相连)，则将左儿子结点加入到子集树中并插入活结点优先队列，否则就不是可行结点。
- 接着继续考察当前扩展结点的右儿子结点。当满足限界条件($cn+n-i > bestn$)时，右子树中可能含有最优解，将右儿子结点加入到子集树中并插入到活结点优先队列中。
- 算法的while循环的终止条件是遇到子集树中的一个叶结点(即 $n+1$ 层结点)成为当前扩展结点。

6.6 最大团问题

//解最大团问题的优先队列式分支限界法

int Clique::BBMaxClique(int bestx[]) {/搜集子集空间树

MaxHeap<CliqueNode> H(1000);

while(i!=n+1)//非叶节点 {

//检查顶点i与当前团中其他顶点之间是否有边相连

bool OK = true;

for(int j=i-1; j>0; B=B->parent,j--) {

if(B->LChild && a[i][j]==0) {

OK = false; break; } }

if(OK) {/左儿子节点为可行结点

if(cn+1>bestn) bestn = cn + 1;

AddLiveNode(H,cn+1,cn+n-i+1,i+1,E,true);}

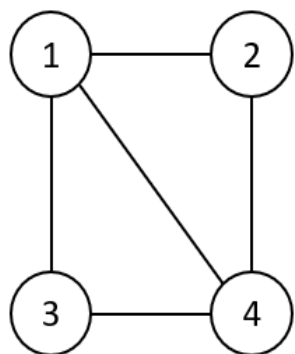
if(cn+n-i>=bestn) { //右子树可能含有最优解

AddLiveNode(H,cn,cn+n-i,i+1,E,false); }

//取下一扩展节点 (略) } }

搜索过程

- cn:当前已包含在顶点集中的顶点个数
- bestn:当前已找到包含最多的顶点个数

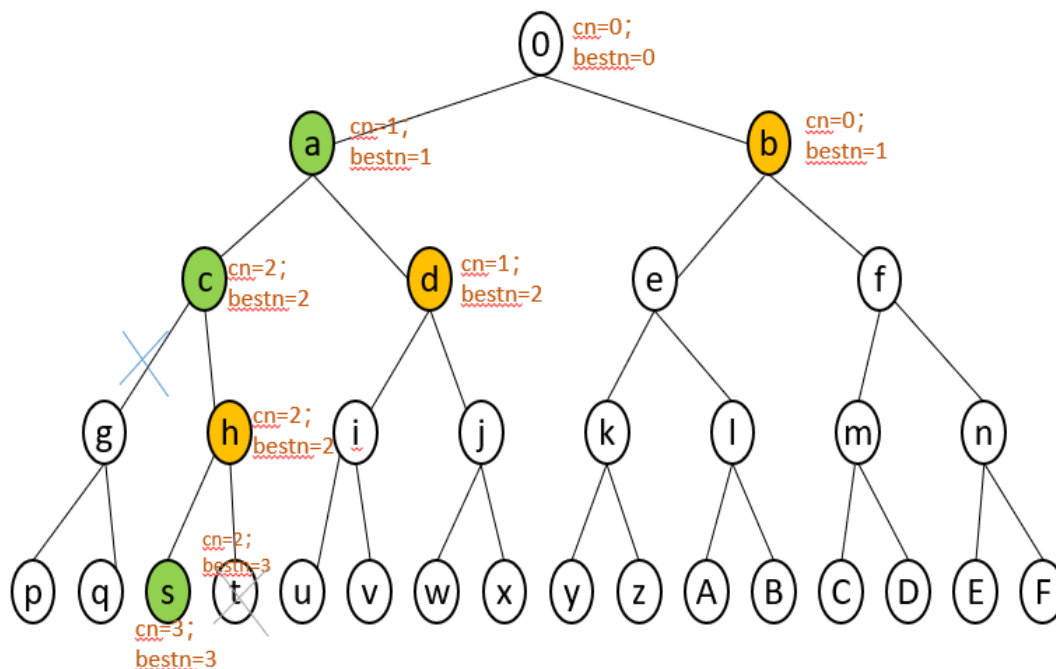


顶点1

顶点2

顶点3

顶点4



$i = 1$

$i = 2$

$i = 3$

$i = 4$

$i = 5$

优先队列元素变化: $\{\} \rightarrow \{\mathbf{a}(4), b(3)\} \rightarrow \{\mathbf{c}(4), b(3), d(3)\} \rightarrow$
 $\{\mathbf{b}(3), d(3), h(3)\} \rightarrow \{\mathbf{d}(3), h(3), e(3), f(2)\} \rightarrow$
 $\{\mathbf{h}(3), e(3), i(3), f(2), j(2)\} \rightarrow \{\mathbf{s}(3), e(3), i(3), f(2), j(2)\}$

■ 通过应用范例学习分支限界法的设计策略。

(1) 装载问题;

(2) 0-1背包问题;

子集树

(3) 最大团问题;

(4) 旅行售货员问题;

排列树

(5) 单源最短路问题;

(6) 批处理作业调度问题

分支限界法限界函数的设计

- 限界函数是分支限界法的核心，其设计直接影响算法最优性和效率。
- 限界函数起到了两方面的作用：

分支限界法限界函数的设计

- 限界函数是分支限界法的核心，其设计直接影响**算法最优性和效率**。
 - 作为**剪枝函数**，要确保剪枝不会丢失潜在的最优解（最优性保证）
 - **最小化问题**：限界函数值 $\leq$ 真实最优解值（下界）
 - **最大化问题**：限界函数值 $\geq$ 真实最优解值（上界）
 - 以装载问题为例，限界函数：当前已装袋的物品重量加上剩下的物品重量

分支限界法限界函数的设计

- 限界函数是分支限界法的核心，其设计直接影响算法最优性和效率。
 - 作为**优先级函数**，尽可能接近真实值（效率）
 - 界限越紧，剪枝效率越高；松散的界限会导致大量无效分支无法被剪除
 - 以0-1背包为例，用贪心法计算剩余物品的最大价值上界
 - 限界函数的计算复杂度应显著低于直接求解问题，否则失去剪枝意义

分支限界法限界函数的设计

- 限界函数是分支限界法的核心，其设计直接影响算法最优性和效率。

问题	限界函数设计
装载问题	当前载重量+剩余集装箱重量
0-1背包问题	当前价值+ 贪心法估算剩余价值
最大团问题	当前在团顶点数+剩余顶点数

这类问题相对简单，优先级分支限界法在找到第一个叶子节点时，即可结束并返回最优解。

分支限界法中堆的设计

- 堆（优先队列）是分支限界法的核心数据结构，负责存储内容，需平衡状态完整性与空间效率。

0-1背包问题(回顾)

准备工作：声明了一个最大堆，来实现活结点优先队列

```
MaxHeap<MaxHeapNode> H(1000); //定义最大堆容量为1000
```

堆中元素类型为MaxHeapNode。子集树中节点类型为bbnode

```
class MaxHeapNode
{
    private:
        Typep upprofit,           //节点的价值上界
        profit;                     //节点所相应的价值
        Typew weight;               //节点所相应的重量
        int level;                 //活节点在子集树中所处的层序号

        bbnode *ptr;               //指向活节点在子集中相应节点的指针
};
```

分支限界法中堆的设计

- 堆（优先队列）是分支限界法的核心数据结构，负责存储内容，需平衡状态完整性与空间效率。

信息类型	必需性	示例问题	存储内容
基础状态	必选	所有问题	当前目标值（如 cc/profit）、层级（level）
限界值	必选	所有问题	lcost（TSP）、bound（背包）
路径历史	条件必选	TSP、作业调度	已访问城市序列（E.x[0:s]）
父节点指针	可选	需输出完整解的问题	parent 指针

■ 通过应用范例学习分支限界法的设计策略。

(1) 装载问题;

(2) 0-1背包问题;

子集树

(3) 最大团问题;

(4) 旅行售货员问题;

排列树

(5) 单源最短路问题;

(6) 批处理作业调度问题

为什么分支限界算法不讨论时间复杂度?

6.7 旅行售货员问题

■ 1. 问题描述

设有 n 个城市组成的交通图，一个售货员从驻地城市出发，到其它城市各一次去推销货物，最后回到驻地城市。假定任意两个城市 i, j 之间的距离 d_{ij} ($d_{ij}=d_{ji}$)是已知的，问应该怎样选择一条最短（或总旅费最小）的路线？

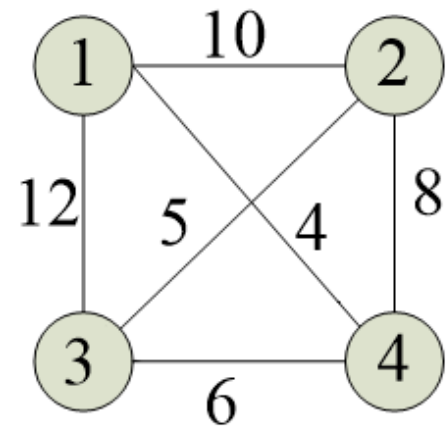
■ 定义问题的解空间

□ 解的形式 $(x_1, x_2, \dots, x_n)$

□ 令 n 个城市组成的集合

$S = \{1, 2, \dots, n\}$ ，则 $\underline{x_1 = 1}$ ，

$x_i \in S - \{x_1, x_2, \dots, x_{i-1}\}$ ， $i = 2, \dots, n$ 。



6.7 旅行售货员问题

■ 限界条件

限界条件(回溯法)

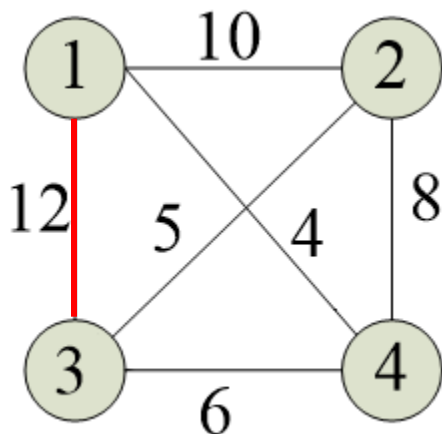
$$cc < bestc$$

- $cc + rcost < bestc$

- cc : 当前**已走过**的城市所用的路径长度

- $rcost$: 当前**未走过**的所有顶点最小出边费用和

- $bestc$: 表示当前找到的最短路径的路径长度



已走过的城市{1,3}

$$cc = 12$$

$$rcost = 5 + 4$$

■ 节点的优先级定义为: 下界 $lcost = cc + rcost$

有没有更紧的界?

准备工作——创建一个最小堆

- 算法开始时创建一个最小堆，用于表示活结点优先队列。(用一个最小优先队列来记录活节点)，队列中每个节点的类型为MinHeapNode。

```
Class MinHeapNode{  
    Type lcost,    //子树费用下界  
    cc,           //当前费用  
    rcost;        //x[s:n-1]中顶点最小出边费用和  
    *x;           //当前路径的顶点排列  
    int s,         //标记搜索进度，已访问x[0:s]  
}
```

MinOut记录图中每个顶点的
最小出边费用

```
MinHeap<MinHeapNode> H(1000); //定义最小堆容量为1000
```

准备工作——计算MinOut

// 计算MinOut[i] = 顶点i的最小出边费用

```
Type *MinOut = new Type [n+1];
```

```
MinSum = 0; //最小出边费用和
```

为什么要提前
算好MinOut?

```
for (int i = 1; i <= n; i++) {
```

```
    Type Min = NoEdge;           // NoEdge是无边的标志
```

```
    for (int j = 1; j <= n; j++)
```

```
        if (a[i][j] != NoEdge && (a[i][j] < Min || Min == NoEdge))
```

```
            Min = a[i][j];
```

```
    if (Min == NoEdge) return NoEdge;    // 此路不通，无解
```

```
    MinOut[i] = Min;
```

```
    MinSum += Min;    }
```

准备工作——初始化

// 把E-节点初始化为树根

MinHeapNode E;

E.x = new int [n];

for (i = 0; i < n; i++)

 E.x[i] = i + 1; //存储当前路径的顶点排列

E.s = 0; //根节点到当前节点的路径

E.cc = 0; // 当前费用为0

E.rcost = MinSum; //顶点最小出边费用和

bestc = NoEdge; // 目前没有找到旅行路径

算法

```
while (E.s < n - 1) { // 搜索排列树, 不是叶子
    if (E.s == n - 2) { // 叶子的父节点, 再添加两条边即完成
        // 检查新的旅行路径是不是更好
        if (a[E.x[n-2]][E.x[n-1]] != NoEdge && a[E.x[n-1]][1] != NoEdge
            && (E.cc + a[E.x[n-2]][E.x[n-1]] + a[E.x[n-1]][1] < bestc || bestc == NoEdge))
            // 找到更优的旅行路径
            { bestc = E.cc + a[E.x[n-2]][E.x[n-1]] + a[E.x[n-1]][1];
              E.cc = bestc;
              E.lcost = bestc;
              E.s++;
              H.Insert(E); }
        else delete [] E.x;
    } // 舍弃扩展结点
```

算法

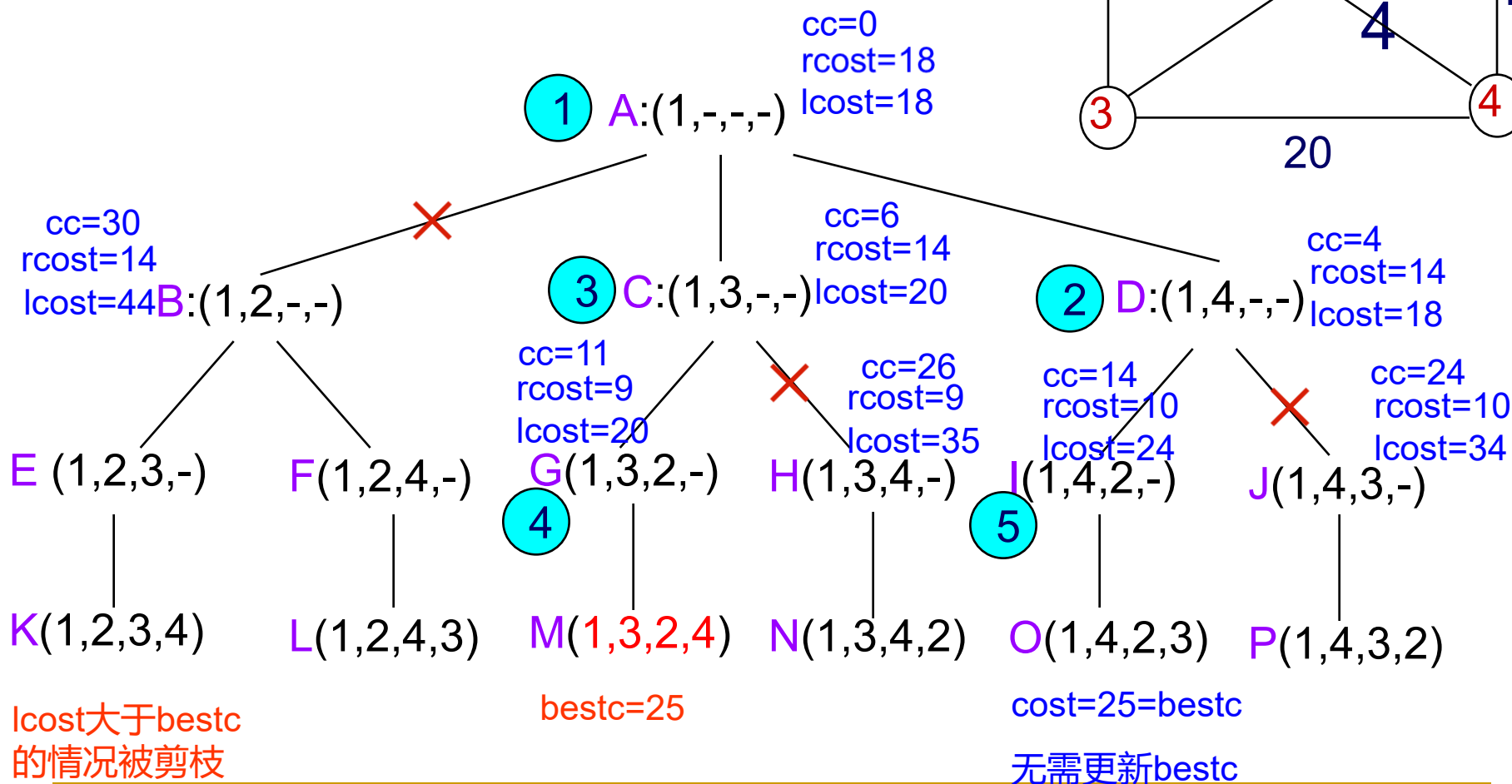
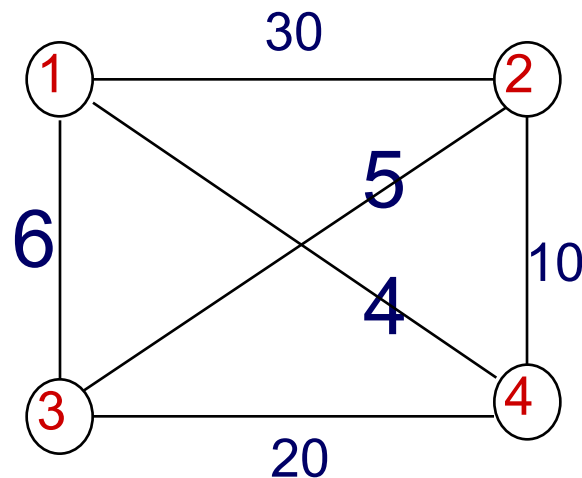
```
else { //s<n-2时:
for (int i = E.s + 1; i < n; i++)
    if (a[E.x[E.s]][E.x[i]] != NoEdge) { // 可行条件判断
        cc = E.cc + a[E.x[E.s]][E.x[i]];
        rcost = E.rcost - MinOut[E.x[E.s]];
        b = cc + rcost; //下限
        if (b < bestc || bestc == NoEdge) { // 有更好的叶子, 保存到堆中
            MinHeapNode N;
            N.x = new int [n];
            for (int j = 0; j < n; j++)    N.x[j] = E.x[j];
            N.x[E.s+1] = E.x[i]; N.x [i]= E.x[E.s+1]; //通过交换枚举出所有排列
            N.cc = cc; N.s = E.s + 1; N.lcost = b; N.rcost = rcost;
            H . Insert(N) ;}
        } // 扩展下一个节点, 直到堆为空(略)
    }
```

结果处理

```
if (bestc == NoEdge) return NoEdge; // 没有旅行路径
// 将最优路径复制到v[1:n] 中
for (i = 0; i < n; i++)
    v[i+1] = E.x[i];
while (true) { // 释放最小堆中的所有节点
    delete [] E.x;
    try { H.DeleteMin(E); } // 取下一个E-节点
    catch (OutOfBounds) { break; } // 没有未处理的节点
}
return bestc;
```

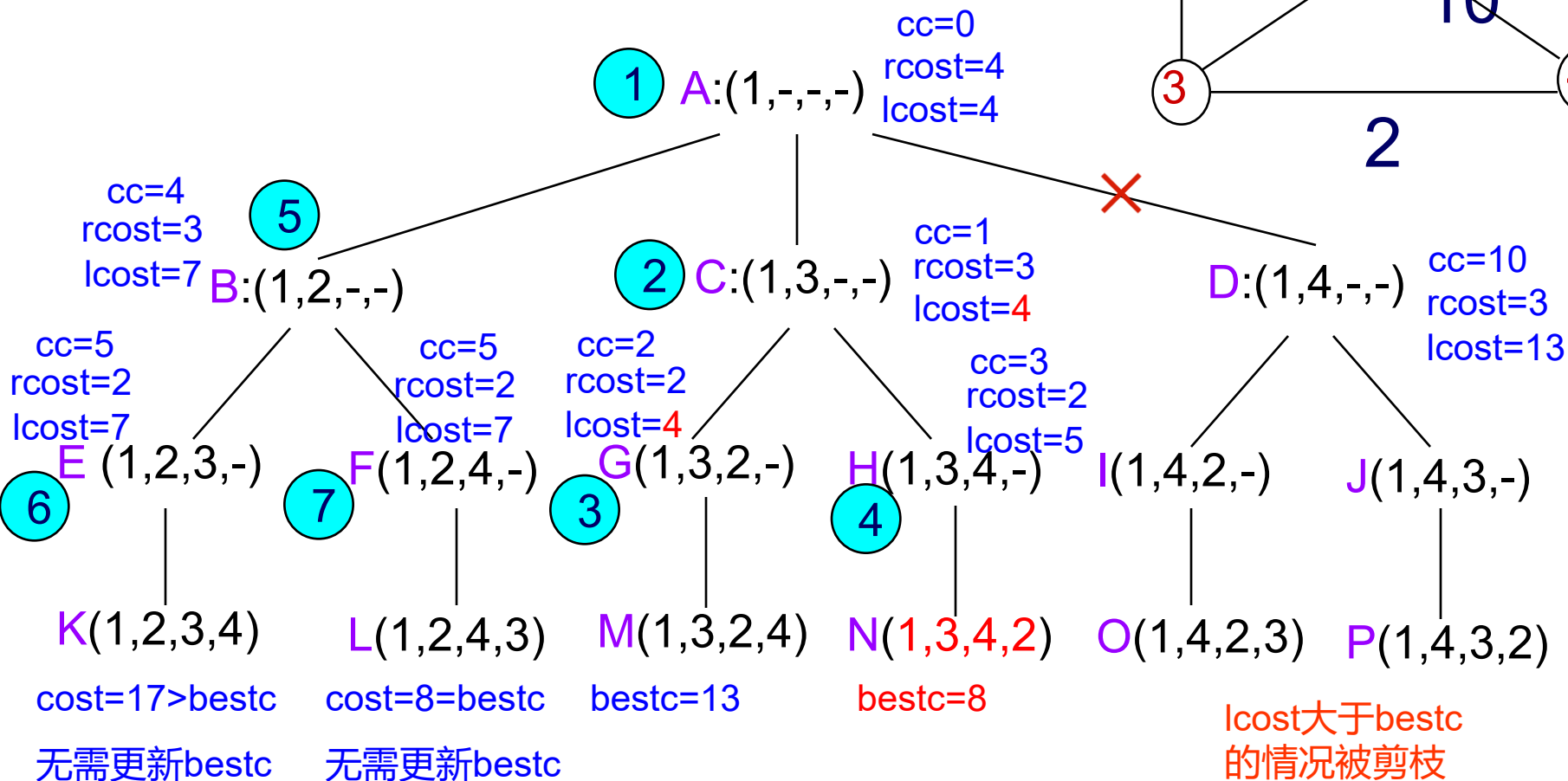
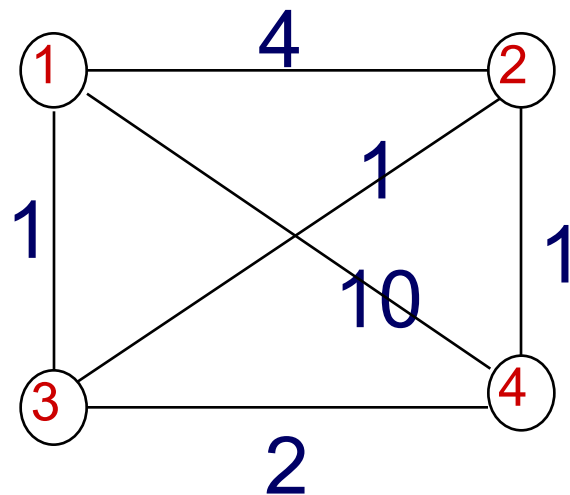
TSP问题 优先队列式分支限界法

优先队列式分支限界法



TSP问题

优先队列式分支限界法



- **通过应用范例学习分支限界法的设计策略。**

- (1) 装载问题;**

- (2) 0-1背包问题;**

- (3) 最大团问题;**

- (4) 旅行售货员问题;**

- (5) 单源最短路问题;**

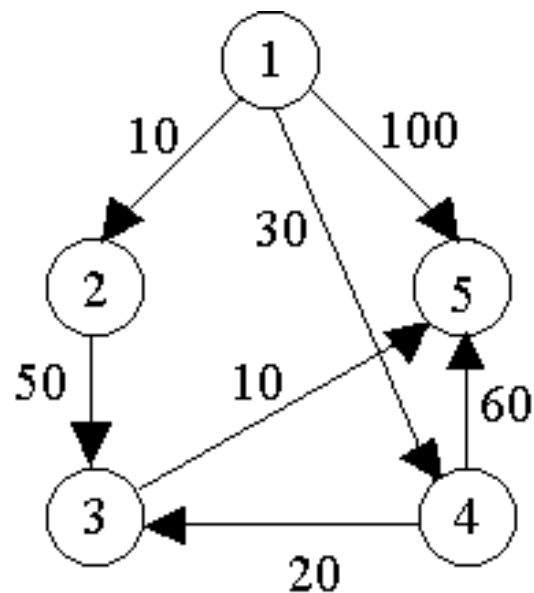
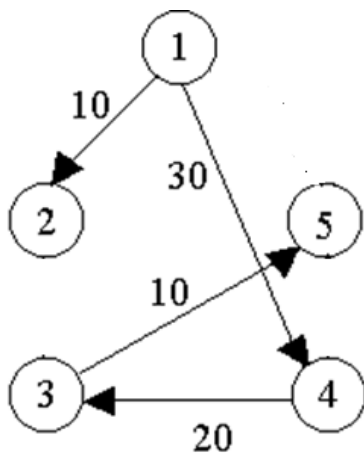
- (6) 批处理作业调度问题**

4.5 单源最短路径

单源最短路径问题：

给定带权有向图 $G = (V, E)$ ，其中每条边的权是非负实数。给定 V 中的一个顶点，称为源。计算从源到所有其它各顶点的最短路长度。长度是指路上各边权之和。

Dijkstra算法是解单源最短路径问题的贪心算法。



6.2 单源最短路径问题

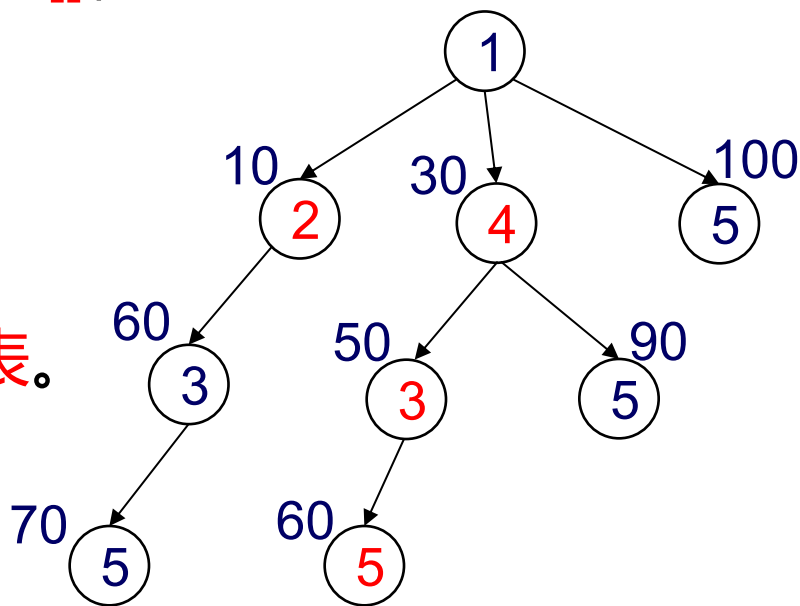
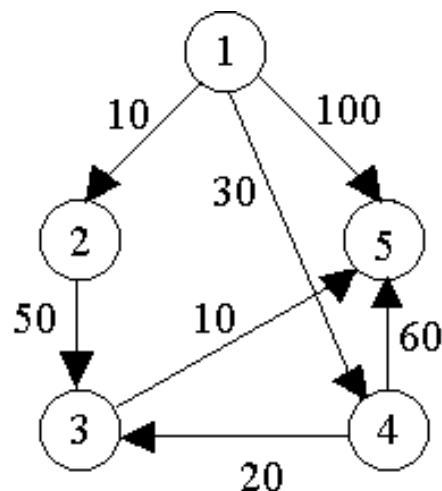
1. 优先级

优先队列式分支限界法解有向图G的单源最短路径问题产生的**解空间树**。通过动态维护一个全局的**最短距离dist[]**，实时更新各顶点的最短距离。

■解单源最短路径问题的优先队列式分支限界法**用最小堆来存储活结点表**。

优先级是结点所对应的当前路长。

E.length



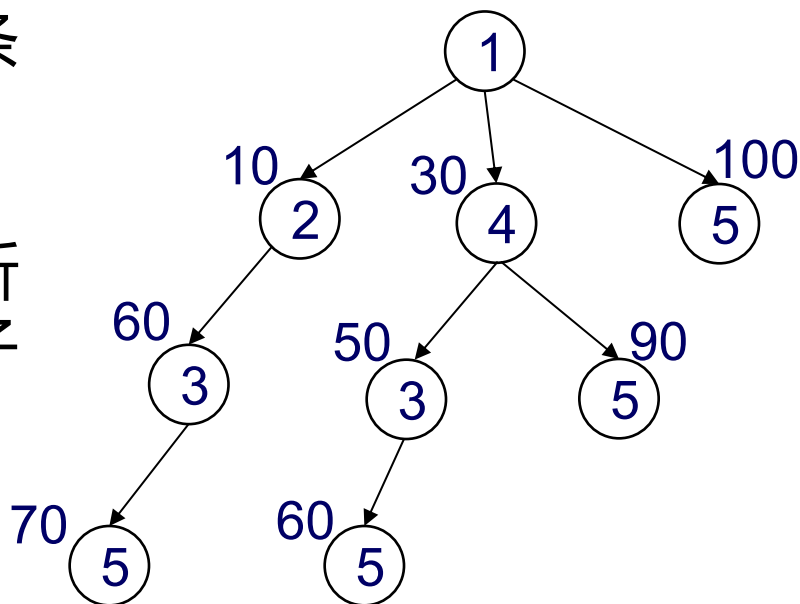
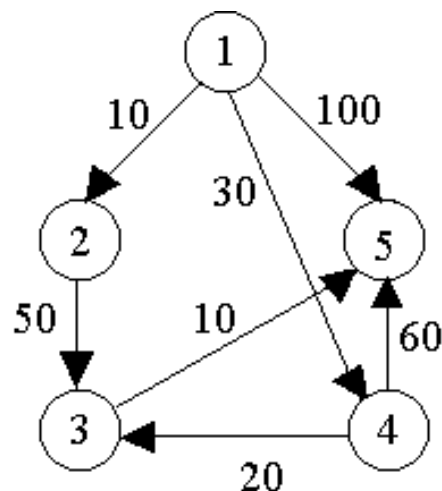
解空间树和常见的子集树、排列树的不同之处？

6.2 单源最短路径问题

2. 剪枝策略

- 约束条件：有边相连
- 限界条件：
 - 从源顶点1出发，可能有多条不同路径到达的同一顶点。由于两条路径的路长不同，因此可以将路长长的路径所对应的树中的结点为根的子树剪去

$$E.length + c[E.i][j] < dist[j]$$



可以用类似于TSP的MinOut吗?

6.2 单源最短路径问题

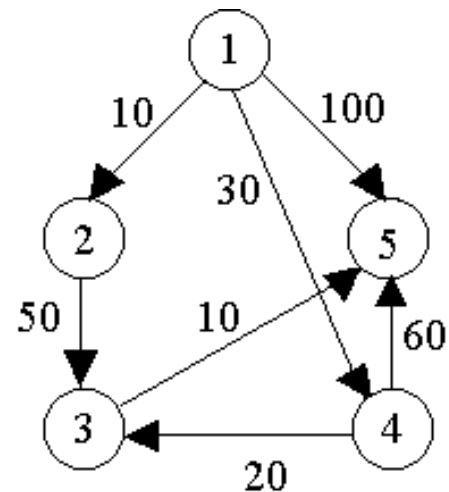
3. 数据结构

Class Graph

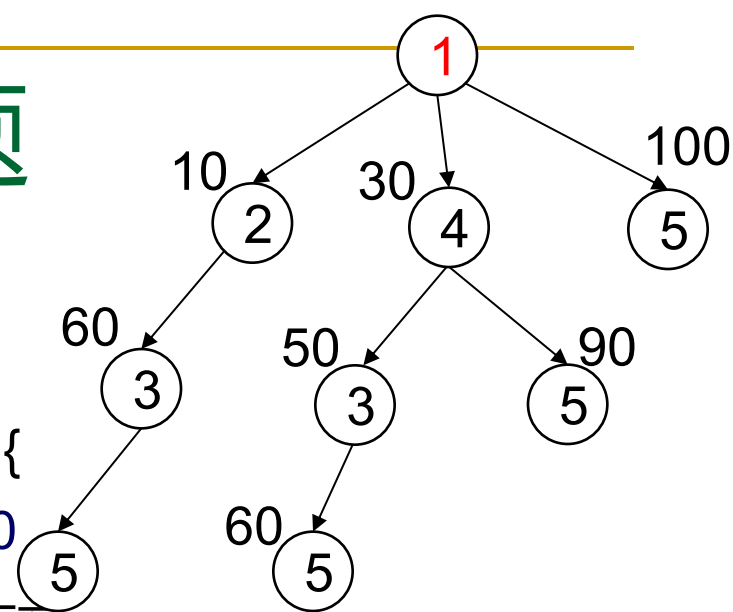
```
{  int  n;    //顶点编号
    Type **c; //邻接矩阵
    Type *dist; //源点到各顶点最短距离
    int *prev; //前驱顶点数组
}
```

Class MinHeapNode

```
{
    int i;          //顶点编号
    Type length;    //当前路长
}
```



6.2单源最短路径问题



```
while (true) {  
    for (int j = 1; j <= n; j++) { //n是顶点个数  
        if ((c[E.i][j]<inf)&&(E.length+c[E.i][j]<dist[j])) {  
            // 顶点i到顶点j可达, 且满足限界约束 70  
            dist[j]=E.length+c[E.i][j];    //更新j的最短距离  
            prev[j]=E.i;    //prev前驱顶点数组  
            // 加入活结点优先队列  
            MinHeapNode<Type> N;  
            N.i=j;  
            N.length=dist[j];    //结点所对应的当前路长  
            H.Insert(N);} }  
    try {H.DeleteMin(E);}    // 取下一扩展结点  
    catch (OutOfBounds) {break;} // 优先队列空}
```

```
{1(0)}  
{2(10),4(30),5(100)}  
{4(30),3(60),5(100)}  
{3(50),3(60),5(90),5(100)}  
{3(60),5(60), 5(90),5(100)}  
{5(60),5(70), 5(90),5(100)}  
{5(70),5(90),5(100)}  
{5(90),5(100)}  
{5(100)}  
}
```

6.2 单源最短路径问题

优先队列分支限界法 VS Dijkstra算法

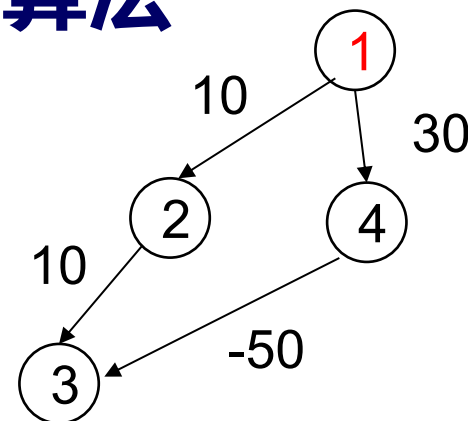
相似之处：两者都用优先队列和dist数组

不同之处：分支限界法不保证节点首次出

队时已获得最短路径（可能重新入队），

而Dijkstra的节点出队后即确定最短路径

。



{1(0)}
{2(10),4(30)}
{3(20),4(30)}
{4(30)}
{3(-20)}
{}

6.2 单源最短路径问题

优先队列分支限界法 VS Dijkstra算法

方法	允许环	处理负权环	适用场景
Dijkstra	否	不支持	仅非负权图
Bellman-Ford	否	能检测但无解	含负权边但无负权环
分支限界法	是	能检测并终止	含环、负权边或复杂约束的变种问题

- **通过应用范例学习分支限界法的设计策略。**

- (1) 装载问题;**

- (2) 0-1背包问题;**

- (3) 最大团问题;**

- (4) 旅行售货员问题;**

- (5) 单源最短路问题;**

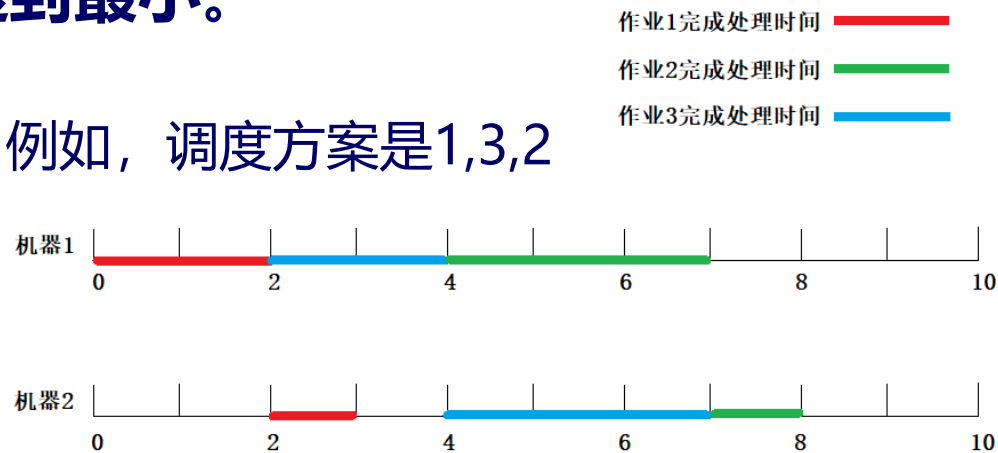
- (6) 批处理作业调度问题**

6.9 批处理作业调度

- 给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业必须先由机器1处理，然后由机器2处理。作业 J_i 需要机器 j 的处理时间为 t_{ji} 。对于一个确定的作业调度， F_{ji} 是作业 J_i 在机器 j 上完成处理时间，所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。
- 批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小。

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

例如，调度方案是1,3,2



$$F_{21} = 3 \quad F_{22} = 7 \quad F_{23} = 8$$

调度方案是1,3,2的完成时间18

6.9 批处理作业调度

■ 批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小。

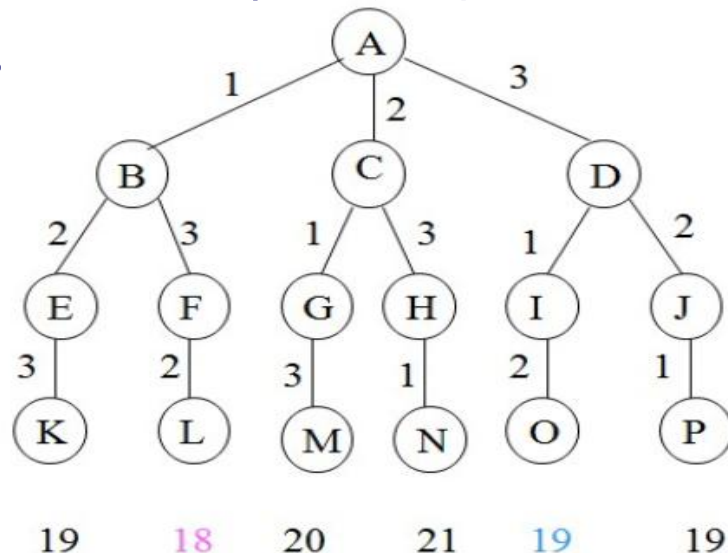
■ 1. 问题的解空间：排列树

□ 解的形式 $(x_1, x_2, \dots, x_n)$

□ x_i 取值范围

$S = \{1, 2, \dots, n\}, x_i \in S - \{x_1, x_2, \dots, x_{i-1}\}$

□ 回溯限界条件：当前部分排列的作业在机器2上的完成时间和 f < 当前找到的最优排列所需要的完成时间和
 $bestf$

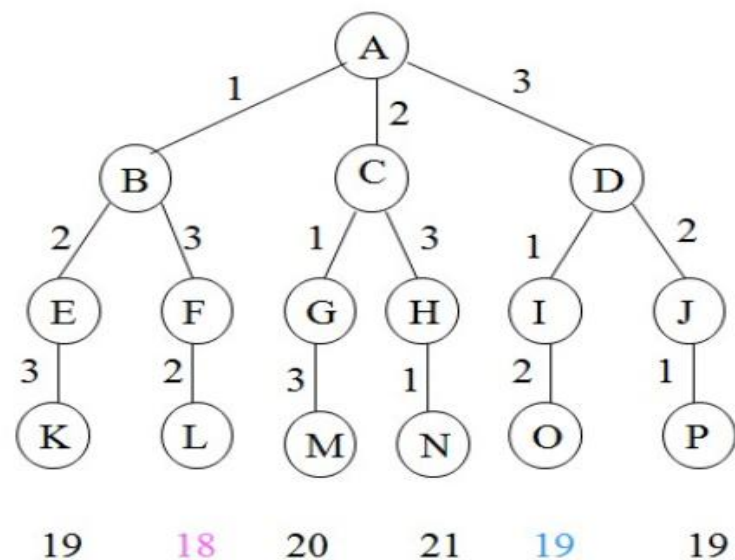
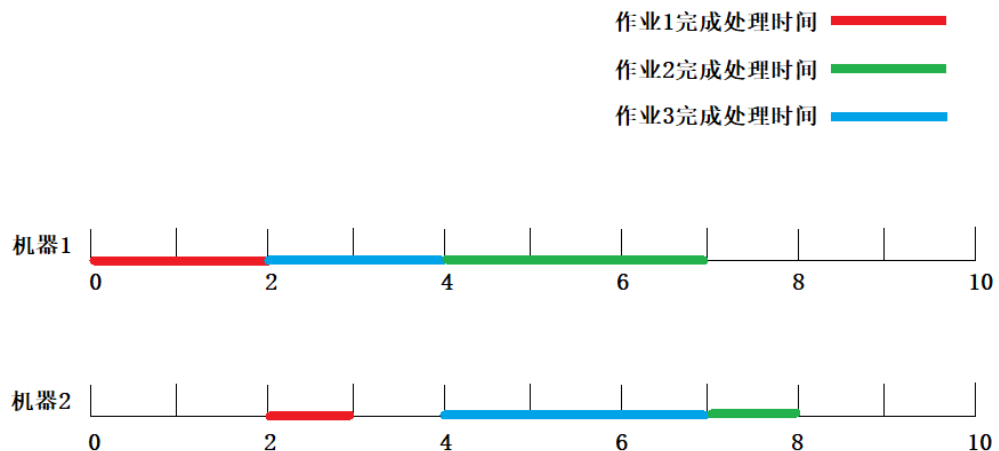


有没有更紧的界？

2. 限界函数Bound

假设在某内部结点处已安排 r 个作业，作业集为 M ，相应

子树中叶结点完成时间和 $f = \sum_{i \in M} F_{2i} + \sum_{i \notin M} F_{2i}$



2. 限界函数Bound



情况1：机器1无空闲时间



- **假设：**机器1连续处理未调度作业，无闲置。
- **剩余作业完成时间和 S_1 ：**

$$S_1 = \sum_{k=r+1}^n [F_{1p_r} + (n - k + 1)t_{1p_k} + t_{2p_k}]$$

○ 解释：

- F_{1p_r} ：最后一个已调度作业在机器1的完成时间。
- $(n - k + 1)t_{1p_k}$ ：作业 p_k 在机器1的处理时间被后续所有未调度作业等待。
- t_{2p_k} ：作业 p_k 在机器2的处理时间。

- **优化：**若 t_{1p_k} 按非减序排列， S_1 取得极小值 $\hat{S}_1$

2. 限界函数Bound



情况2：机器2无空闲时间



- **假设：** 机器2连续处理未调度作业，无闲置。
- **剩余作业完成时间和 S_2 ：**

$$S_2 = \sum_{k=r+1}^n \left[\max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + (n - k + 1)t_{2p_k} \right]$$

○ **解释：**

- $\max(F_{2p_r}, F_{1p_r} + \min t_{1i})$ ：机器2开始处理第一个未调度作业的时间（需等待机器1完成至少 $\min t_{1i}$ ）。
- $(n - k + 1)t_{2p_k}$ ：类似 S_1 的等待时间。

○ **优化：** 若 t_{2p_k} 按非减序排列， S_2 取得极小值 $\hat{S}_2$

2. 限界函数Bound

假设在某内部结点处已安排 r 个作业，作业集为 M ，相应

子树中叶结点完成时间和 $f = \sum_{i \in M} F_{2i} + \sum_{i \notin M} F_{2i}$

- 如果机器1没有空闲：

$$S_1 = \sum_{k=r+1}^n [F_{1p_r} + (n - k + 1)t_{1p_k} + t_{2p_k}]$$

- 如果机器2没有空闲：

$$S_2 = \sum_{k=r+1}^n \left[\max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + (n - k + 1)t_{2p_k} \right]$$

剩余任务完成时间和的下界是：

$$f \geq \sum_{i \in M} F_{2i} + \max \{ \hat{S}_1, \hat{S}_2 \}$$

作为优先队列式分支限界法中的限界函数Bound。

6.9 批处理作业调度问题

3. 算法描述

```
while (E.s <= n) {  
    if (E.s == n) { // 叶结点  
        if (E.sf2 < bestc) { //更新当前最优值bestc和相应的最优解bestx  
            bestc = E.sf2;  
            for (int i = 0; i < n; i++)  
                bestx[i] = E.x[i];  
            delete [] E.x;  
        }  
    }  
}
```

E.s=n的情形，当前扩展结点E是排列树中的叶结点。E.sf2是相应于该叶结点的完成时间和。当E.sf2 < bestc时,更新当前最优值bestc和相应的当前最优解bestx。

6.9 批处理作业调度问题

3. 算法描述

else { // 产生当前扩展结点的儿子结点

```
    for (int i = E.s; i < n; i++) {
```

```
        Swap(E.x[E.s],E.x[i]);
```

```
        int f1,f2;
```

```
        int bb=Bound(E,f1,f2,y);
```

```
        if (bb < bestc ) {
```

```
            MinHeapNode N;
```

```
            N.NewNode(E,f1,f2,bb,n);
```

```
            H.Insert(N);}
```

```
        Swap(E.x[E.s],E.x[i]);
```

```
    }
```

```
delete [] E.x;} // 完成结点扩展
```

$E.s < n$ 时，算法依次产生当前扩展结点E的所有儿子结点。对于当前扩展结点的每一个儿子结点node，计算出其相应的完成时间和的下界bb。当 $bb < bestc$ 时，将该儿子结点插入到活结点优先队列中。而当 $bb \geq bestc$ 时，可将结点node舍去

小 结

- 分支限界法是在解空间树上带剪枝的宽度优先搜索算法。分为队列式和优先队列式两种分支限界法
- 与回溯法解决问题的类型相似，但解决问题的方法不同。
- 算法复杂性分析

习题：

- 要求：给出问题分析、解空间、剪枝函数、算法思想文字描述、和算法伪代码
- （1）教材 算法实现题6-4最小重量机器设计问题
- （2）教材 算法实现题6-8 最佳调度问题
- 在canvas上提交